Projeto Final DevSecOps Clube Aurora (Turma 1746)

> **Guia Geral do Projeto:** Este documento apresenta a estrutura completa do laboratório final, o contexto do caso corporativo, as diretrizes de escolha entre os três níveis de entrega e as instruções gerais para execução.

---

0. Resumo em 1 Minuto

O **Projeto Final do Clube Aurora** foi estruturado em três níveis de entrega independentes. Escolha a opção que melhor se alinha à sua bagagem técnica e aos seus objetivos profissionais no curso:

Nível	Perfil Ideal	Tempo Estimado	Arquivos Entregues	Nota Máxima	Pré-requisitos Técnicos
Nível 1 (Teórica Guiada)	Análise conceitual, governança e conformidade	~4 h	`RELATORIO_FINAL.md`	10,0	Navegador web e editor de texto (sem terminal/Docker)
Nível 2 (Mista)	Equilíbrio entre código e modelagem conceitual	~6 h	Pull Request (F1, F2, F4) + `EVIDENCIAS.md` + `RELATORIO_TEORICO.md`	10,0	Git básico, conta GitHub (saber abrir PR e ler logs)
Nível 3 (Técnica)	Engenharia DevSecOps, automação e correção total	~6 h a 8 h	Pull Request (F1..F7) + `.trivyignore` + `EVIDENCIAS.md`	10,0	Git, fork, CI/CD e raciocínio de debugging

> **Regra de Ouro da Avaliação:**
> **Todos os três níveis permitem alcançar a nota máxima (10,0).** A avaliação não privilegia código em detrimento da teoria: o rigor na análise de risco, a qualidade das justificativas e a clareza técnica são pontuados com o mesmo peso. Escolha o nível em que você mais vai aprender!

---

1. Como Escolher seu Nível

Para tomar uma decisão rápida e segura, responda com sinceridade a estas três perguntas:

1. **Você já clonou um repositório, criou uma branch e abriu um Pull Request no GitHub?**
   - *Se não:* O **Nível 1 (Teórica Guiada)** foi feito sob medida para você. Você trabalhará com relatórios pré-gerados e templates guiados, sem risco de travar em comandos de terminal.
   - *Se sim:* Você tem a base necessária para o **Nível 2** ou **Nível 3**.
2. **Você se sente confortável abrindo a aba Actions para entender a saída de um erro no console?**
   - *Se isso parece complexo ou desconfortável:* O **Nível 1** ou o **Nível 2** (que conta com passo a passo ilustrado) oferecem uma jornada segura e sem ansiedade.
   - *Se você quer exercitar a leitura de logs de segurança (SCA, SAST, DAST):* Escolha o **Nível 2** ou **Nível**

3.

3. **Qual é o seu foco prioritário de aprendizado no momento?**
   - *Modelagem de ameaças (STRIDE), conformidade, LGPD e governança executiva:* Escolha o **Nível 1** ou **Nível 2**.
   - *Engenharia de pipeline, correção de vulnerabilidades em código e contêineres:* Escolha o **Nível 3**.

Desmistificando o Nível 3 (Técnica)

Se você quer tentar o **Nível 3**, não tenha receio de ficar travado(a):
* O projeto oferece **pistas progressivas** (veja `GUIA_NIVEL_3_TECNICO.md`) para cada falha.
* O repositório conta com um script de autochecagem da **professora** que você pode rodar a qualquer momento para saber exatamente o que falta.
* **A pontuação parcial é real:** se você corrigir 5 ou 6 falhas e documentar no relatório o que tentou fazer nas demais, sua nota será proporcional e muito alta (consulte a `RUBRICA.md`).

1.1 Regra de Transição Fluida entre Trilhas & Aterrissagem Segura

A arquitetura do projeto foi desenhada de forma **modular, cumulativa e concêntrica**, permitindo que você mude de trilha com total liberdade e sem perder nada do que já produziu:

```
                NÍVEL 3 (TÉCNICA TOTAL)
  Código F1 a F7 + Relatório de Evidências

                NÍVEL 2 (MISTO)
    Código F1, F2, F4 + Teórico F3, F5, F6, F7

              NÍVEL 1 (TEÓRICA GUIADA)
       Análise Conceitual F1 a F4
```

1. **Promoção (Upgrade N1 N2 N3):**
   - Se você começou no **Nível 1** e, ao entender a lógica das falhas, sentiu confiança para colocar a mão na massa, basta clonar o repositório e implementar as correções de código de **F1, F2 e F4**. Você migra automaticamente para o **Nível 2**!
   - Se no **Nível 2** você conseguir avançar também para **F3, F5, F6 e F7** no código, submeta o Pull Request completo e migre para o **Nível 3**. Todo o raciocínio e documentação já elaborados continuam válidos.

2. **Aterrissagem Segura (Fallback N3 N2 N1 sem penalidade):**
   - Se você optou pelo **Nível 3**, mas encontrou restrições de tempo ou dificuldades em falhas dinâmicas mais avançadas (como os headers HTTP no ZAP ou a orquestração do Compose), **não se desespere e não abandone a entrega!**
   - Você pode realizar uma **aterrissagem segura no Nível 2**: submeta o código das falhas que conseguiu sanar (ex: F1, F2 e F4) e complete as análises conceituais das demais no relatório `TEMPLATE_TM.md`. Você concorrerá à **nota máxima (100 pts / 10,0)** dentro dos critérios do Nível 2!
   - Da mesma forma, se preferir focar exclusivamente na governança e modelagem de ameaças, você pode optar pela entrega do `TEMPLATE_TG.md` como **Nível 1**, também concorrendo à nota máxima.

3. **Como formalizar a sua escolha final:**
   - Não há necessidade de comunicação prévia ou burocracia: sua trilha é definida pelo template de entrega que você submeter na plataforma e no Pull Request (`TEMPLATE_TG.md` para Nível 1,

`TEMPLATE_TM.md` para Nível 2 ou `TEMPLATE_EVIDENCIAS.md` para Nível 3). Marque a caixa de seleção correspondente no topo do documento.

---

2. O Caso Clube Aurora

2.1 Contexto Corporativo

O tio **Armando Lacerda** é um empresário carismático e fundou recentemente o **Clube Aurora**, uma startup de concierge de luxo que gerencia viagens em jatos privativos, compras em leilões fechados e reservas exclusivas para indivíduos de altíssimo patrimônio (*Ultra High Net Worth Individuals UHNWI*).

Para colocar a empresa no ar com rapidez máxima, Armando contratou seu sobrinho de 21 anos, **Leonardo ("Léo")**, estudante de computação talentoso e autodidata, para construir toda a plataforma digital do zero em Python e Flask.

O modelo de negócio foi um sucesso instantâneo! Em poucos meses, o volume de transações e reservas disparou. Diante do crescimento acelerado, o tio Armando contratou uma nova gerente de atendimento, **Carla Nogueira**, e fez dois pedidos urgentes para Léo:
1. *"Léo, me dê acesso de administrador geral do sistema, e dê admin para a Carla também, para ela resolver qualquer problema dos clientes na hora."*
2. *"Não invente burocracia nem barreiras de segurança que atrasem a inclusão de novas funcionalidades. Precisamos de velocidade total!"*

Sobrecarregado e sem mentoria em arquitetura corporativa, Léo implementou soluções rápidas: comitou chaves de API e senhas no repositório, não configurou headers de segurança, utilizou consultas diretas em banco e deixou o papel de administrador como padrão para novos cadastros.

Preocupado com o peso da responsabilidade e com a estabilidade da empresa, Léo chamou **você** especialista em DevSecOps para auditar o sistema, estancar os riscos imediatos e implementar uma esteira de segurança contínua antes que ocorra um vazamento desastroso.

2.2 Notícia Fictícia (O Risco Real de Negócio)

> [NOTÍCIA] **PORTAL DEFESA DIGITAL 22/10/2026**
> **"Vazamento massivo expõe agenda, transações e residências de clientes VIP em concierge de luxo na Europa"**
> *Uma violação crítica de segurança expôs os dados confidenciais de mais de 1.200 clientes de altíssima renda de uma plataforma de concierge suíça. Investigadores forenses apontam que credenciais de produção estavam versionadas em arquivos de configuração no repositório Git, combinadas com bibliotecas desatualizadas com falhas de Request Smuggling e injeção de SQL nos campos de busca. Os atacantes também escalaram privilégios no servidor aproveitando contêineres Docker executados como root com sockets administrativos expostos. Especialistas alertam que multas pesadas de proteção de dados (GDPR/LGPD) e processos civis de clientes VIP podem forçar o fechamento imediato da empresa.*

---

3. Mapa das 7 Falhas Deliberadas (F1 a F7)

A aplicação do Clube Aurora possui 7 vulnerabilidades e más práticas didáticas distribuídas em código, contêiner e orquestração. A tabela abaixo resume cada uma delas e como provar que foram solucionadas:

ID	Vulnerabilidade / Desafio	Onde se localiza	Ferramenta que aponta hoje	Nível que corrige no código	Como saber que você resolveu
F1	Segredos de produção no `.env` e rastreamento Git	`aurora/.env`, `aurora/.gitignore`	Revisão de código / Gitleaks	Nível 2 e Nível 3	`git ls-files .env` retorna vazio; `.env` listado no `.gitignore`; variáveis de exemplo criadas em `.env.example`
F2	Dependências antigas com CVEs Críticos/Altos (SCA)	`aurora/requirements.txt`	Trivy Filesystem (SCA)	Nível 2 e Nível 3	Job `Trivy FS` fica verde no GitHub Actions (`Total: 0 (HIGH: 0, CRITICAL: 0)`)
F3	SQL Injection via interpolação de strings (SAST)	`aurora/app/db.py` (`search_clients`)	Semgrep SAST	Nível 3	Job `Semgrep` fica verde (`0 findings`) e busca de clientes funciona com prepared statement (`?`)
F4	Dockerfile sem hardening (root, tag mutable)	`aurora/Dockerfile`	Hadolint (Lint) e Trivy Image	Nível 2 e Nível 3	Hadolint sem alerta `DL3007`; contêiner executa como usuário restrito (`USER nonroot`); Trivy Image verde
F5	Ausência de Security Headers HTTP essenciais	`aurora/app/main.py` e `aurora/app/__init__.py`	OWASP ZAP (DAST)	Nível 3	Invocação local `curl -I` exibe os 4 cabeçalhos de segurança (CSP, HSTS, X-Content, X-Frame); ZAP sem alertas FAIL
F6	Quebra de Menor Privilégio e elevação de papel	`aurora/app/db.py` e `aurora/app/main.py`	Revisão / validador da professora	Nível 3	Novo usuário criado recebe papel `staff` por padrão; rota de alteração de papel exige admin e barra usuários comuns com HTTP 403

ID	Vulnerabilidade / Desafio	Onde se localiza	Ferramenta que aponta hoje	Nível que corrige no código	Como saber que você resolveu
F7	Infraestrutura como Código permissiva (`privileged: true`)	`aurora/docker-compose.yml`	Revisão de IaC	Nível 3	sem `docker-compose.yml` `privileged: true` , sem socket `/var/run/docker.sock` montado e sem portas expostas indevidamente

---

4. Nível 1 Teórica Guiada

4.1 Perfil e Visão Geral

Desenhado para estudantes que desejam focar em **análise de risco, modelagem de ameaças e tomada de decisão estratégica em cibersegurança**. Você atuará como o(a) consultor(a) executivo(a) que recebe os relatórios técnicos prontos e precisa traduzir os riscos técnicos para o Tio Armando.

4.2 O Que Você Entrega

- Um único arquivo: **`RELATORIO_FINAL.md`** , criado a partir do preenchimento estruturado do `aluno/TEMPLATE_TG.md` .

4.3 O Que Você NÃO Precisa Fazer

- Não precisa instalar Python, Docker, Git ou extensões complexas.
- Não precisa fazer fork, clonar repositórios ou executar comandos em terminal.
- Não precisa alterar código-fonte nem abrir Pull Requests.
- Todos os dados de evidência (tabelas do Trivy, saídas do Semgrep e Hadolint) já estão disponibilizados no diretório `evidencias_pipeline/` .

4.4 Passo a Passo Resumido

1. **Passo 1 (Leitura do Cenário):** Leia o caso do Clube Aurora (Seção 2) e o caso F1 resolvido como exemplo guia no `TEMPLATE_TG.md` .
2. **Passo 2 (Modelagem STRIDE & CIA):** Consulte as evidências pré-geradas e preencha a classificação STRIDE e os impactos na Tríade CIA para as falhas F2, F3 e F4.
3. **Passo 3 (Decisão de Governança):** Analise o pedido de admin do Tio Armando (F6) aplicando os conceitos de Segregação de Funções (SoD) e Menor Privilégio.
4. **Passo 4 (Comunicação Executiva):** Redija o memorando de 3 parágrafos para o Tio Armando explicando a gravidade dos riscos e os próximos passos sem jargões indecifráveis.

[NOTA] **Pistas Progressivas para o Nível 1 (Clique para expandir)** * **Pista para F2 (SCA):** Observe que as falhas no `urllib3` e `gunicorn` envolvem *Request Smuggling* e negação de serviço. Como isso ameaça a Disponibilidade (A) e a Integridade (I) de um leilão de luxo em tempo real? * **Pista para F3 (SQLi):** Na busca de clientes, se um invasor injetar `' OR 1=1 --`, o que acontece com a lista de clientes VIP? Trata-se de uma ameaça direta de *Information Disclosure* (I do STRIDE) violando a Confidencialidade (C). * **Pista para F6 (Pedido do Armando):** Conceder acesso administrativo irrestrito à gerente Carla cria risco de fraude

interna e impede trilhas de auditoria idôneas. Uma solução profissional é criar um papel intermediário (ex: `manager` ou `concierge_lead`) com permissões estritamente necessárias para a operação, sem acesso a logs ou configurações do sistema.

4.5 Critério de Pronto (Checklist de Autoavaliação)

- [ ] Todas as seções do `TEMPLATE_TG.md` preenchidas sem deixar nenhum campo `[A preencher]`.
- [ ] Classificação STRIDE e CIA justificada com base nos dados reais do caso.
- [ ] Resposta ao pedido do Tio Armando baseada em governança e mitigação de fraude interna.
- [ ] Memorando executivo claro, objetivo e estruturado em exatamente 3 parágrafos.

4.6 Armadilhas Comuns a Evitar

- *Copiar definições de dicionário:* Evite explicar o que é "S" do STRIDE de forma abstrata; explique como o Spoofing afeta especificamente os clientes VIP do Clube Aurora.
- *Tentar rodar o código localmente:* Não perca tempo configurando ambiente local se você escolheu o Nível 1. Use diretamente as evidências fornecidas.

---

5. Nível 2 Mista

5.1 Perfil e Visão Geral

Ideal para quem busca equilíbrio: você coloca a mão na massa para corrigir as falhas mais comuns do dia a dia (segredos, dependências e contêineres) e exercita a visão arquitetural nas falhas mais conceituais de aplicação.

5.2 O Que Você Entrega

1. **Pull Request no GitHub:** Contendo as correções no código para **F1** (segredos e `.gitignore`), **F2** (`requirements.txt`) e **F4** (`Dockerfile`).
2. `EVIDENCIAS.md`**:** Preenchido a partir do `aluno/TEMPLATE_EVIDENCIAS.md` com links do PR e prints dos checks verdes.
3. `RELATORIO_TEORICO.md`**:** Preenchido a partir do `aluno/TEMPLATE_TM.md` cobrindo a modelagem teórica de **F3, F5, F6 e F7**.

5.3 O Que Você NÃO Precisa Fazer

- Não precisa mexer em SQL complexo ou query parametrizada de banco (F3).
- Não precisa escrever código de cabeçalhos HTTP no Flask (F5).
- Não precisa alterar lógica de autenticação e papéis no Python (F6).
- Não precisa alterar o arquivo `docker-compose.yml` (F7).
- Não precisa mudar o Quality Gate para `MEDIUM`.

5.4 Guia Dedicado com Pistas Progressivas

Consulte o guia completo com dicas técnicas e exemplos comentados em:
`aluno/GUIA_NIVEL_2_MISTO.md`

[NOTA] **Pistas Rápidas para as Correções Práticas do Nível 2 (Clique para expandir)** * **Pista F1 (Segredos):** Execute `git rm --cached aurora/.env` para parar de rastrear o arquivo sem apagá-lo do disco. Depois, certifique-se de que `.env` está em `aurora/.gitignore` e crie `aurora/.env.example` com valores

ilustrativos. * **Pista F2 (SCA):** No `aurora/requirements.txt`, atualize para versões sem CVEs conhecidos: `Flask==3.0.3`, `Werkzeug==3.0.3`, `gunicorn==23.0.0` e `urllib3==2.8.0`. * **Pista F4 (Dockerfile):** Substitua `FROM python:latest` por uma imagem estável como `python:3.11-slim`. Crie um usuário sem privilégios com `RUN useradd -u 10001 -m nonroot` e declare `USER nonroot` antes do `ENTRYPOINT`.

5.5 Passo a Passo Resumido

1. **Passo 1:** Faça o Fork do repositório, ative o GitHub Actions e crie a branch `feature/correcoes-mista`.
2. **Passo 2:** Remova o `.env` do rastreamento do Git e configure o `.gitignore` (F1).
3. **Passo 3:** Atualize as versões vulneráveis no `requirements.txt` (F2).
4. **Passo 4:** Aplique os mandamentos no `Dockerfile` (multi-stage build e usuário `nonroot`) (F4).
5. **Passo 5:** Suba os commits, verifique os jobs Hadolint e Trivy FS verdes no GitHub Actions e preencha os relatórios.

5.6 Critério de Pronto

- [ ] Pull Request aberto e apontando para a branch `main` do seu fork.
- [ ] Job `1. Quality Gate Estático` aprovado nos steps de Hadolint e Trivy FS.
- [ ] `RELATORIO_TEORICO.md` preenchido com a análise de STRIDE/CIA de F3, F5, F6 e F7.
- [ ] `EVIDENCIAS.md` preenchido com link do PR e prints comprobatórios.

5.7 Armadilhas Comuns a Evitar

- *Esquecer de desindexar o `.env`:* Apenas colocar `.env` no `.gitignore` não remove o arquivo se ele já estiver comitado. É preciso rodar `git rm --cached .env`.
- *Subir dependências quebradas:* Sempre verifique se os testes básicos continuam passando (`pytest` ou conferência manual).

---

6. Nível 3 Técnica

6.1 Perfil e Visão Geral

Para quem busca imersão profunda em Engenharia DevSecOps: resolução ponta a ponta das 7 falhas, automação completa de pipeline, teste dinâmico com DAST e gerenciamento profissional de exceções de vulnerabilidade.

6.2 O Que Você Entrega

1. **Pull Request Completo:** Corrigindo todas as 7 falhas deliberadas (**F1 a F7**).
2. **Disparo com Quality Gate em MEDIUM:** Disparo manual via `workflow_dispatch` com sensibilidade aumentada.
3. **Registro de Exceção em `.trivyignore`:** Documentação técnica de aceitação de risco temporária conforme boas práticas.
4. **`EVIDENCIAS.md`:** Preenchido a partir do `aluno/TEMPLATE_EVIDENCIAS.md` com todas as saídas e relatórios comprovados.

6.3 Guia Dedicado com Estratégia de Resolução

Consulte a documentação aprofundada, roteiros locais e troubleshooting em:
`aluno/GUIA_NIVEL_3_TECNICO.md`

[NOTA] **Pistas Rápidas para as Falhas Avançadas do Nível 3 (Clique para expandir)** * **Pista F3 (SQL Injection):** Em `app/db.py`, altere `get_db().execute(f"... LIKE '%{name}%' ...")` para prepared statement: `get_db().execute("SELECT ... WHERE full_name LIKE ? ORDER BY id", (f"%{name}%",))`. * **Pista F5 (Security Headers):** Registre os cabeçalhos HTTP com `@bp.after_request` em `app/main.py` e com `@app.after_request` em `app/__init__.py` adicionando `Content-Security-Policy: default-src 'self'`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` e `Strict-Transport-Security: max-age=31536000; includeSubDomains`. * **Pista F6 (Menor Privilégio):** Em `app/db.py`, mude o valor padrão para `def create_user(..., role="staff")`. Em `app/main.py`, proteja o endpoint com `@bp.post("/admin/users//role")` seguido imediatamente por `@auth.require_role("admin")`. * **Pista F7 (IaC):** Em `docker-compose.yml`, remova `privileged: true`, remova o mapeamento do volume `/var/run/docker.sock`, feche a porta `5000` e carregue variáveis com `env_file: .env`.

6.4 Ordem Crítica de Resolução (Evitando Bloqueios)

No pipeline do Clube Aurora, a esteira opera em modo Fail-Fast. Siga esta ordem para otimizar seu tempo:
1. **F2 (Dependências)** + **F3 (SQL Injection)** + **F4 (Dockerfile)**: *Essas 3 falhas liberam a construção da imagem contêiner e o scanner estático.*
2. **F1 (Segredos)** e **F7 (Compose)**: *Saneamento de credenciais e infraestrutura local.*
3. **F6 (Menor Privilégio)**: *Garante que os testes de autorização passem no validador.*
4. **F5 (Headers HTTP)**: *Configura o ZAP Baseline para validar a aplicação no ar com status verde.*

6.5 Critério de Pronto

- [ ] Validador automatizado (`validate.py` ou `validate.ps1`) retornando **7 PASSes (100% OK)**.
- [ ] Testes automatizados (`pytest -v`) passando sem regressão.
- [ ] Todos os jobs do GitHub Actions verdes (estático e dinâmico, incluindo ZAP Baseline).
- [ ] Disparo com gate `MEDIUM` documentado e arquivo `.trivyignore` preenchido.
- [ ] `EVIDENCIAS.md` preenchido com link do PR e prints dos resultados.

6.6 Armadilhas Comuns a Evitar

- *Inversão de decoradores no Flask:* O decorador `@bp.post(...)` deve envolver a rota primeiro; a restrição `@auth.require_role("admin")` fica logo abaixo.
- *Tentar consertar o ZAP com a imagem quebrada:* O ZAP só roda se o contêiner subir com sucesso. Corrija o Dockerfile e as dependências primeiro!

---

7. Preparando o Ambiente no GitHub

Para quem escolheu o **Nível 2 (Mista)** ou **Nível 3 (Técnica)**, siga este passo a passo para configurar seu ambiente de trabalho no GitHub:

Passo 1: Criar o Fork do Repositório

1. Acesse o repositório oficial do projeto informado pela **professora** no GitHub.
2. No canto superior direito da página, clique no botão **Fork**.
3. Em **Repository name**, mantenha o nome padrão ou defina o de sua preferência.
4. Escolha a visibilidade desejada:
   - **Público (Recomendado):** Permite acesso completo à aba nativa **Security / Code Scanning** do GitHub, com alertas visuais integrados.

- **Privado (100% Suportado):** O pipeline foi projetado para funcionar perfeitamente em repositórios privados, exibindo tabelas de vulnerabilidades diretamente no console de execução e disponibilizando os relatórios para download na seção **Artifacts**. Caso seu repositório seja privado, lembre-se de adicionar a **professora** como colaboradora na entrega.

5. Clique em **Create fork**.
   * Como saber que deu certo:* Você será redirecionado para `https://github.com/SEU-USUARIO/devsecops-projeto-final-aurora` com a indicação *"forked from ..."*.

Passo 2: Habilitar o GitHub Actions no seu Fork

Por padrão de segurança do GitHub, os workflows em repositórios criados por fork vêm pausados.
1. No seu repositório no GitHub, clique na aba **Actions**.
2. Clique no botão verde: **"I understand my workflows, go ahead and enable them"**.
* Como saber que deu certo:* A aba Actions deixará de exibir a tela de aviso e passará a mostrar o workflow `Pipeline DevSecOps - Clube Aurora`.

> **Nota sobre Permissões:** O workflow deste projeto já declara explicitamente as permissões mínimas necessárias (`permissions: contents: read, security-events: write`). Você **não** precisa alterar as permissões gerais de Actions em Settings para "Read and write".

Passo 3: Clonar o Repositório e Criar sua Branch

1. Abra seu terminal (ou Git Bash / VS Code) e clone o repositório:
   `bash git clone https://github.com/SEU-USUARIO/devsecops-projeto-final-aurora.git cd devsecops-projeto-final-aurora`
2. Crie uma branch de trabalho isolada (nunca trabalhe direto na `main`):
   `bash git checkout -b feature/correcoes-seguranca`
   * Como saber que deu certo:* O comando `git branch` destacará `* feature/correcoes-seguranca`.

Passo 4: O Primeiro Disparo (Vendo os Quality Gates Reprovarem)

1. Envie a branch recém-criada para o GitHub para disparar o pipeline:
   `bash git push -u origin feature/correcoes-seguranca`
2. Abra seu repositório no GitHub e acesse a aba **Actions**.
3. Você verá o workflow em execução e, em seguida, **FALHANDO** com status vermelho ([X]) no Job 1 (SAST, SCA e Lint) e no Job 2 (Container SCA).
   * Como saber que deu certo:* O pipeline falha com `exit code 1` apontando vulnerabilidades reais nos logs. Essa é a comprovação inicial (o estado "vermelho") necessária para o seu relatório!

Passo 5: Abrir o Pull Request Inicial

1. No GitHub, vá até a aba **Pull requests** e clique em **New pull request**.
2. Configure: `base: main` `compare: feature/correcoes-seguranca`.
3. Título sugerido: `[DevSecOps] Correções de Segurança - Clube Aurora`.
4. Clique em **Create pull request**.
   * Como saber que deu certo:* O PR será criado e exibirá os checks automáticos com avisos de bloqueio de merge devido aos Quality Gates reprovados.

---

8. Como Investigar os Relatórios

Para diagnosticar o que precisa ser corrigido, você tem três fontes de informação no GitHub:

1. **Console do GitHub Actions (Mais Rápido e Universal):**
   Clique na execução que falhou na aba **Actions** e expanda os steps vermelhos (ex: `Executar Trivy Filesystem`, `Executar Semgrep SAST` ou `Executar Hadolint`). O console exibe tabelas detalhadas com biblioteca, CVE, linha do arquivo e descrição do problema.
2. **Aba Security / Code Scanning Alerts (Se o repositório for Público):**
   Os resultados são enviados em formato SARIF para a aba de segurança do GitHub, permitindo navegar pelos alertas com filtros visuais de severidade e links diretos para os trechos de código.
3. **Download de Artefatos ZIP (Relatórios Técnicos Completos):**
   Na página principal da execução do workflow, role até a seção **Artifacts** no final da tela para baixar:
   - `relatorios-estaticos-sast-sca`: relatórios detalhados em JSON, SARIF e texto do Hadolint, Trivy e Semgrep.
   - `zap-dast-reports`: relatório completo em HTML e Markdown gerado pelo OWASP ZAP (quando o contêiner estiver aprovado e no ar).

> [ATENÇÃO] **Por Que o ZAP DAST Não Executou no Início?**
> O pipeline adota o princípio de **Fail-Fast**: como a imagem contêiner vulnerável continha múltiplos CVEs de severidade HIGH, o Trivy Image bloqueou a esteira antes de inicializar o contêiner. Isso evita que uma aplicação comprovadamente vulnerável seja exposta na rede. Assim que você corrigir as dependências e o Dockerfile, a aplicação subirá e o ZAP executará normalmente!

Para um guia visual sobre como ler cada coluna dos relatórios de SCA e SAST, consulte: `aluno/COMO_LER_OS_RELATORIOS.md`

---

9. Sugestão de Organização Pessoal de Tempo (Roteiro em 4 Etapas)

> [NOTA] **Nota Importante sobre o Tempo e Mudança de Trilha:**
> A organização abaixo é **exclusivamente uma sugestão de autogestão** para ajudá-lo(a) a estruturar seus estudos (estimada em 4 blocos de ~1,5 h). Ela **não tem relação direta com os dias de aula do curso**. Você tem total autonomia para executar no seu próprio ritmo, e pode **mudar de trilha quando bem entender** ao longo de todo o processo. O que define a sua avaliação é a entrega final que você submeter!

Etapa Sugerida	Nível 1 (Teórica Guiada)	Nível 2 (Mista)	Nível 3 (Técnica)	Marco de Autonomia
Etapa 1	Ler o caso, contexto e glossário; estudar o exemplo F1 no manual	Fork, habilitação do Actions, clonagem local e primeiro disparo (vermelho)	Fork, setup de branch, primeiro disparo e auditoria local	Ambiente pronto e trilha inicial selecionada
Etapa 2	Preencher análise STRIDE e Tríade CIA para F2 e F3	Corrigir F1 (segredos) e F2 (`requirements.txt`); push e teste	Corrigir F2, F3 e F4 (eliminar bloqueios da imagem contêiner)	Primeiros jobs verdes aparecendo no pipeline

Etapa Sugerida	Nível 1 (Teórica Guiada)	Nível 2 (Mista)	Nível 3 (Técnica)	Marco de Autonomia
Etapa 3	Analisar F4 e responder a decisão de governança do Tio Armando (F6)	Corrigir F4 (`Dockerfile`) e redigir teoria de F3 e F5	Corrigir F1, F5, F6 e F7; rodar validador até 100% OK	Aplicação funcional e protegida contra ataques
Etapa 4	Redigir memorando executivo de 3 parágrafos e revisar checklist	Finalizar relatório teórico (F6 e F7) e montar `EVIDENCIAS.md`	Executar gate MEDIUM, documentar `.trivyignore` e finalizar evidências	Entrega final concluída e checklist verificado

---

10. Como Entregar e Como Será Avaliado

10.1 Orientações de Entrega e Prazos

- **O que entregar:** O relatório preenchido e correspondente ao nível escolhido (Nível 1: `TEMPLATE_TG`, Nível 2: `TEMPLATE_TM`, Nível 3: `TEMPLATE_EVIDENCIAS`), acompanhado dos links públicos do repositório/Pull Request para os níveis práticos.
- **Como entregar:** Submeter o documento preenchido (em formato Word, PDF ou Markdown) devidamente identificado com seu nome completo e turma.
- **Prazo final:** Fique atento(a) à data e ao horário limite estipulados pela **professora**.

10.2 Checklist de Verificação Antes do Envio

Antes de submeter seu trabalho, abra e confira item por item do checklist oficial da turma:
`aluno/CHECKLIST_ENTREGA.md`

10.3 Como sua Nota Será Calculada

A avaliação seguirá rigorosamente a rubrica multidimensional da disciplina:
`aluno/RUBRICA.md`

Os principais pilares avaliados são:
1. **Compreensão do Risco e Fundamentação:** Capacidade de explicar a causa-raiz e o impacto no negócio.
2. **Eficácia das Correções ou Análises:** Validação de que os Quality Gates ou as recomendações resolvem o problema.
3. **Clareza na Comunicação:** Organização do Pull Request, clareza das evidências e ausência de ruídos técnicos desnecessários.
4. **Política de Pontuação Parcial:** No Nível 3, falhas que não foram 100% resolvidas mas possuem investigação documentada com hipóteses consistentes recebem pontuação parcial substancial!

---

11. Colaboração e Integridade Acadêmica

- **Discussão Técnica entre Colegas (Incentivada):** Debater abordagens, trocar ideias sobre leitura de logs e discutir conceitos de modelagem STRIDE com seus pares é uma prática recomendada e reflete o trabalho real de um time de DevSecOps.
- **Autoria e Compreensão Individual:** Todos os textos, justificativas, relatórios e Pull Requests devem ser de autoria própria. Cada estudante deve ser capaz de defender e explicar qualquer linha de código ou justificativa submetida.

- **Transparência e Ética:** Não compartilhe soluções prontas com colegas nem faça cópia direta de repositórios alheios. A avaliação valoriza o processo de aprendizado, a evolução das correções e a capacidade analítica.

---

12. Dúvidas Frequentes e Problemas Comuns (FAQ)

FAQ 1: "Corrigi as dependências no `requirements.txt`, mas o Trivy continua apontando vulnerabilidades. O que pode ser?"

Normalmente isso ocorre se alguma biblioteca foi atualizada para uma versão que ainda possui CVEs conhecidos (por exemplo, atualizar o `urllib3` para `2.0.0` em vez da versão corrigida `2.8.0`). Confira a coluna *Fixed Version* na tabela do Trivy no console do Actions ou consulte o COMPARATIVO_SEGURANCA.md para verificar as versões mínimas recomendadas.

FAQ 2: "Meu repositório é privado e a aba Security não exibe os alertas. O pipeline quebrou?"

Não! No plano GitHub Free, repositórios privados não possuem a interface nativa do GitHub Advanced Security (Code Scanning). O pipeline do curso foi configurado especificamente para isso: ele imprime todas as tabelas detalhadas diretamente no console do GitHub Actions e gera arquivos para download na seção **Artifacts**. Sua avaliação não depende da aba Security nativa.

FAQ 3: "O step do OWASP ZAP DAST foi ignorado (skipped). Meu pipeline está com defeito?"

Não. O workflow adota o princípio de *Fail-Fast*: se o contêiner falhar na checagem de vulnerabilidades (Trivy Image), a aplicação não é inicializada para evitar expor um serviço vulnerável à rede. Assim que você corrigir o `requirements.txt` e o `Dockerfile`, a imagem contêiner será aprovada, a API subirá na porta 8080 e o ZAP executará a varredura completa.

FAQ 4: "Posso mudar de trilha se achar o Nível 3 complexo demais ou o Nível 1 simples demais?"

Sim, com total liberdade! Você pode transitar entre as trilhas quando bem entender ao longo de todo o desenvolvimento do projeto. Como a base de conhecimento e a história do Clube Aurora são compartilhadas, você não perde nada do que estudou. O critério de avaliação será determinado exclusivamente pelo pacote final que você decidir entregar.

FAQ 5: "Preciso ter o Docker Desktop instalado na minha máquina física para fazer o Nível 2 ou Nível 3?"

Não é obrigatório! Toda a execução de contêineres e scanners pode rodar integralmente nos servidores do GitHub Actions na nuvem. Você só precisa editar os arquivos no seu editor de código preferido, comitar e subir para o GitHub (`git push`). O Docker local é apenas uma conveniência para quem deseja testar comandos offline com maior rapidez.

FAQ 6: "Se eu travar em uma falha ou conceito específico, onde posso buscar apoio?"

Utilize a estratégia dos 3 níveis de apoio:
1. *Primeiro nível:* Consulte as pistas progressivas no guia da sua trilha (`GUIA_NIVEL_2_MISTO.md` ou `GUIA_NIVEL_3_TECNICO.md`), o `PLANO_DE_TESTES_DEBUGGING.md` e a tabela de erros comuns.
2. *Segundo nível:* Troque ideias e discuta hipóteses técnicas com seus colegas de turma.
3. *Terceiro nível:* Tire dúvidas diretamente com a **professora** durante as aulas ou nos canais de comunicação da disciplina.